

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 1

Basic question 1: Hash tables

You find an open addressing hash table (using linear probing and modular compression) in the following state. The tokens **X** are the result of lazy deletions:

0	1	2	3	4	5	6	7	8	9
Dog		X	Bee	X	Cat			Ant	X

For each element, list every number in the range 0–9 that can be its hash value.

Answer

Element	Possible hash values (0–9)
Ant	8
Bee	2, 3
Cat	2, 3, 4, 5
Dog	8, 9, 0

The easiest way to answer this question is to use the invariant for an open addressing hash table: for every element X , every slot from the hash value $h(X)$ of X to the actual slot of X (wrapping around if needed) is occupied (by another element or a token **X**). So the possible hash values for X are all the indices from the start of the cluster of X to (including) the current index of X .

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 2

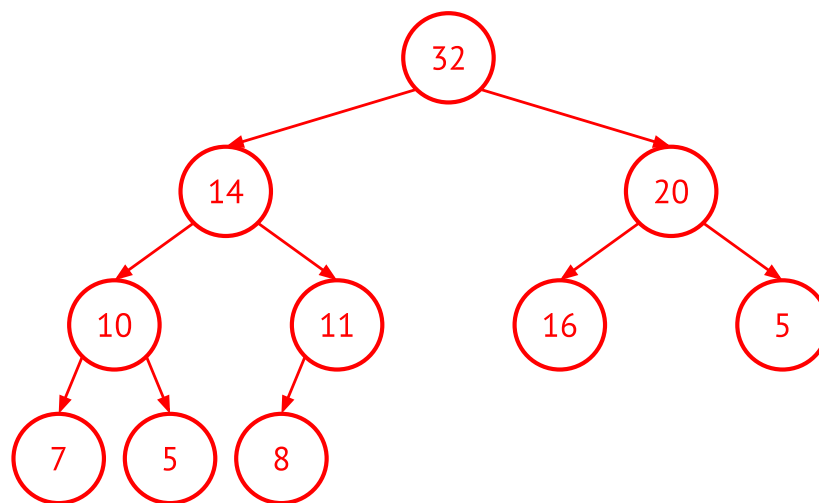
Basic question 2: Heaps

Here is a binary max-heap:

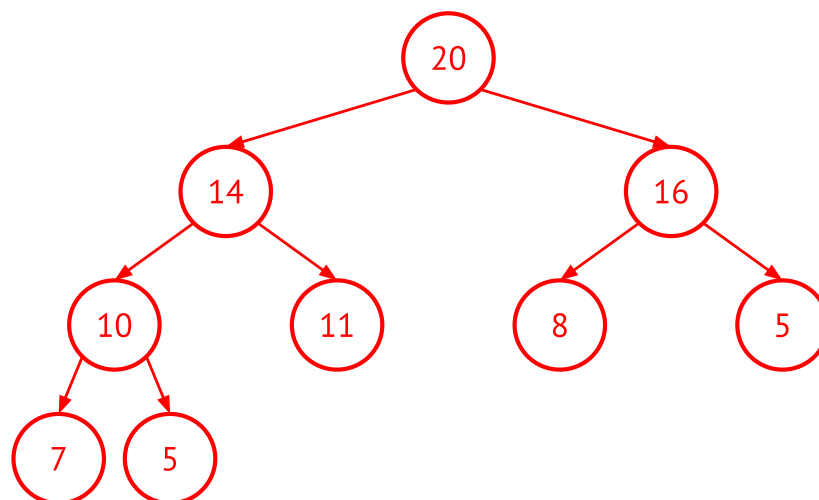
0	1	2	3	4	5	6	7	8	9
32	14	20	10	11	16	5	7	5	8

Answer

- a) Draw the binary max-heap in tree representation.



- b) We run the removeMax operation. Draw the resulting heap in tree representation.



Basic question 3: graphs

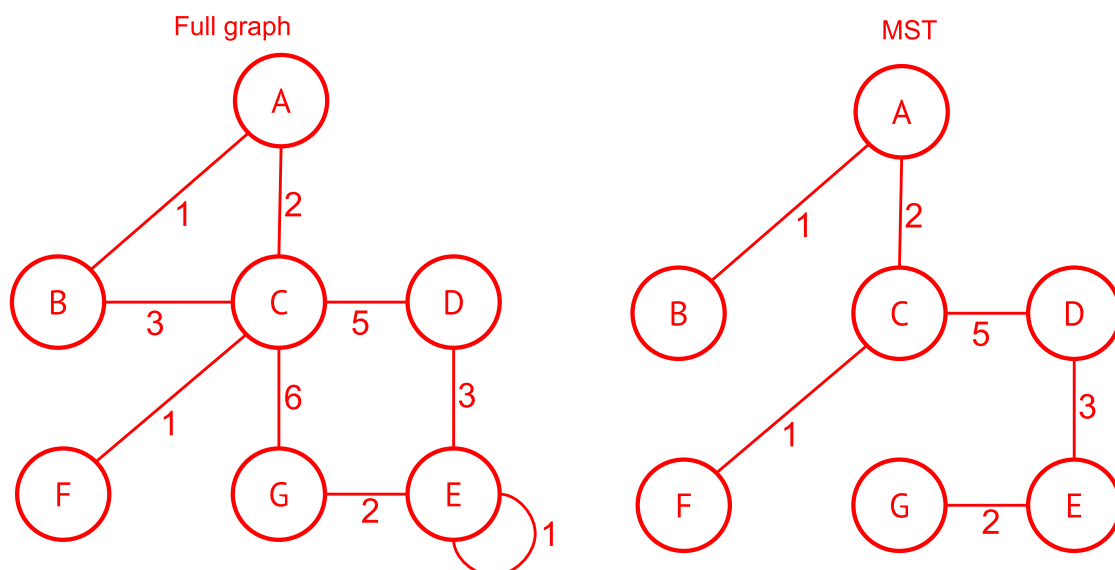
Here is a weighted graph represented as an adjacency matrix.

	A	B	C	D	E	F	G
A		1	2				
B	1		3				
C	2	3		5		1	6
D			5		3		
E				3	1		2
F			1				
G			6		2		

- Draw the weighted graph.
- Draw (or highlight) a minimum spanning tree of the graph.

Lifhack: If you ever find yourself drawing a graph, things may get less messy if you start by putting a node with lots of edges in the middle.

Answer



CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 4

Basic question 4: Merge sort

We do a recursive merge sort of the following 7-element array:

0	1	2	3	4	5	6
M	E	R	G	I	N	G

Note:

- Characters are compared according to their position in the alphabet: $E < G < I < M < N < R$.
- When splitting a subarray of odd length, the left part should be the larger one.
- The recursive call for the left part happens before the recursive call for the right part.

Answer by listing the merge steps in the order in which they are performed by the algorithm.

Use the following format (example): $\text{merge}([A, M], [L, O, S, T]) = [A, L, M, O, S, T]$

Answer

- $\text{merge}([M], [E]) = [E, M]$
- $\text{merge}([R], [G]) = [G, R]$
- $\text{merge}([E, M], [G, R]) = [E, G, M, R]$
- $\text{merge}([I], [N]) = [I, N]$
- $\text{merge}([I, N], [G]) = [G, I, N]$
- $\text{merge}([E, G, M, R], [G, I, N]) = [E, G, G, I, M, N, R]$

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 5

Basic question 5: Complexity

A *streak* in an array is a segment of equal elements. For example, the array [2, 1, 1, 2, 2, 2, 4] has a streak [2, 2, 2] of length 3 (and this is the maximum streak length for this array).

Here is an algorithm (using a helper function) for the maximum streak length:

```

maxStreakLength(array):
    result = 0
    start = 0
    while start < array.length:
        length = streakLength(array, start)
        if result < length:
            result = length
    (1)    start = start + 1
    return result

streakLength(array, start):
    stop = start + 1
    while stop < arr.length and array[start] = array[stop]:
        stop = stop + 1
    return stop - start

```

- Determine the asymptotic complexity of maxStreakLength in the length N of the array.
 - Answer in O -notation, and as tight and simple as possible.
 - Justify your answer (for example by annotating the above pseudocode).
- We replace line (1) with $\text{start} = \text{start} + \text{length}$. What is the asymptotic complexity of maxStreakLength with this modification? (You do not have to justify your answer)

Answer

- The helper function streakLength consists of a loop with $O(1)$ body and at most N iterations, so is $O(N)$. With this, the loop in maxStreakLength has $O(N)$ body and at most N iterations, so is $O(N^2)$.

Why this is tight (not required): we get quadratic behaviour for arrays of equal elements.

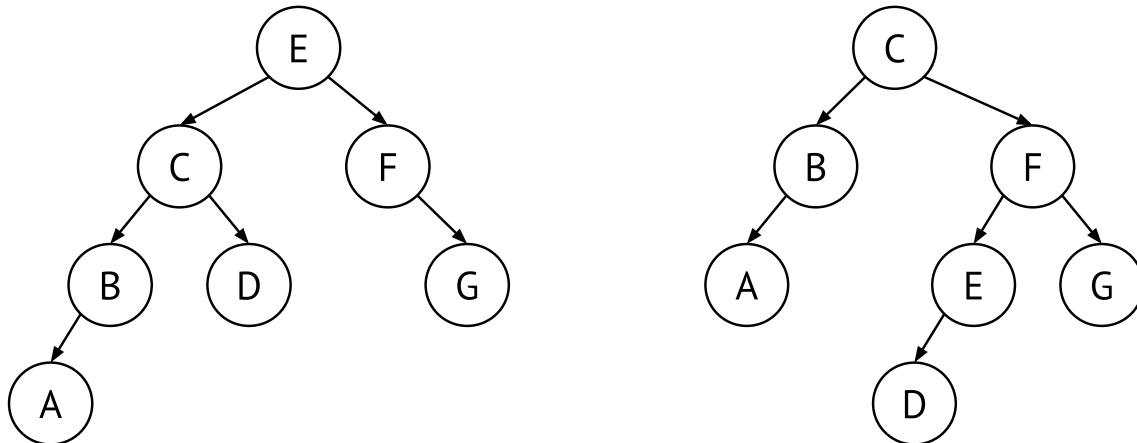
- $O(N)$

Justification (not required, many possibilities): checking the condition of the while-loop in streakLength is a dominant operation of the algorithm (it runs as often as any other line). But with the modification, it runs at most once for each value of stop in $(0, N]$.

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 6

Basic question 6: Tree rotations

Here are two binary search trees containing the same values:



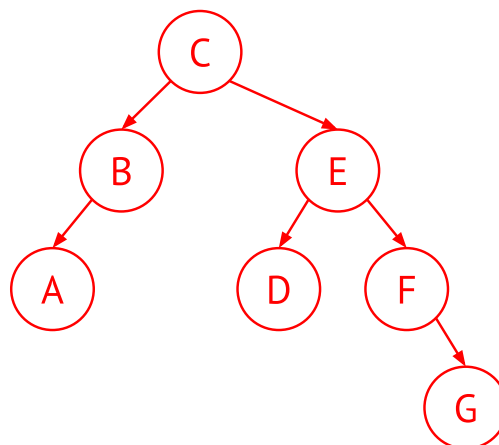
Transform the left tree into the right tree using **two** tree rotations.

- State the rotations in the order you perform them (where in the tree and in what direction).
- Draw the tree after the first rotation.

Answer

- Right rotation at E (with left child C) followed by left rotation at E (with right child F).

b)



CHALMERS	Anonymous code		Consecutive page no L�pande sid nr
	Anonym kod		Question no. Uppgift nr 7

Advanced question 7: Set partition

A *partition* of a set is a decomposition into *parts* (disjoint inhabited subsets).¹

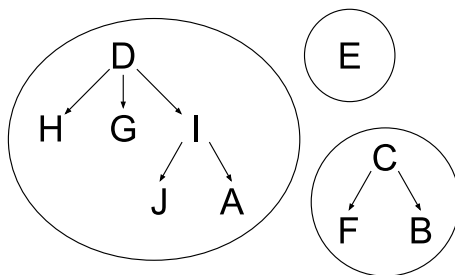
The SetPartition ADT supports the following operations:

```

add(x: Value)                // Add a new element x, forming its own part.
same_part(x: Value, y: Value) → Boolean  // Are the elements x and y in the same part?
join(x: Value, y: Value)      // Merge the parts of elements x and y.

```

To efficiently implement this, we represent the elements as nodes of a (rooted) *forest*: a graph that is the disjoint union of (rooted) trees (which represent the parts).²



Example: a set with 10 elements partitioned into 3 parts (drawn as bubbles), represented as a forest with roots C, D, E

We store the forest using parent pointers (instead of using children pointers as usual):

```

class Node:
    value: Value
    parent: Node    // null for a root node
    size: Integer   // the size of the tree rooted at this node

class Forest implements SetPartition:
    nodes: Map[Value, Node] = initially empty AVL map

    add(x: Value):
        if x is not in nodes:
            nodes.put(x, new Node(x, null, 1))

```

We impose the following balance invariant: every node has at least twice the size of each of its children. For example, in the picture above, node D has size 6 and its largest child I has size 3.

Implement the missing operations `same_part` and `join`. They should run in $O(\log(N))$ where N is the size of the set, assuming values compare in $O(1)$. You do not have to justify this complexity.

Hint: implement a helper method

```
find_root(x: Value) → Node.
```

¹ Mathematicians call it an *equivalence relation* on the set; the parts are the *equivalence classes*.

² Note that the same partition can be represented by different forests.

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 8

Answer

```

find_root(x: Value) → Node
    n = nodes.get(x)
    while n.parent is not null:
        n = n.parent
    return n

same_part(x: Value, y: Value) → Boolean
    return find_root(x).value == find_root(y).value

join(x: Value, y: Value):
    u = find_root(x)
    v = find_root(y)
    if not u.value == v.value:
        // Enforce u.size ≤ v.size.
        if not u.size ≤ v.size: swap u and v

        // Add u as child of v.
        u.parent = v
        v.size = v.size + u.size

```

The following notes are not required by the question.

Here is why all the operations of our set partition implementation run in $O(\log(N))$. As we go up a path in the forest, the node size at least doubles each step (by the balance invariant), up to at most N . Therefore, every such path has length at most $\log_2(N)$. This shows $\text{find_root}(x)$ is $O(\log(N))$. Interacting with the AVL map nodes is also $O(\log(N))$. All other parts are $O(1)$.

The set partition data type is useful in Kruskal's algorithm for a minimum spanning tree in a graph. The algorithm maintains a spanning forest of the graph. Going over the edges by order of weight, it adds an edge to the spanning forest unless it would create a cycle. But how do we check this? Using a set partition data type! We start by adding all nodes as elements. Whenever we consider an edge for the spanning forest, we test if source and target are in the same part (that would mean a cycle). If not, we add it and join the two parts in the set partition.

Caveat: the spanning forest maintained by Kruskal is **not** the forest in the set partition data type. Their trees have the same nodes, but different edges. Two different forests in the same algorithm!

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 9

Advanced question 8: Game data structure

You got hired as a junior programmer to work on MonsterHorde™. In this game, a player is fighting a (non-empty) horde of enemy monsters. Every monster has a number of *hitpoints* that decrease when the player attacks and increase when a monster is restored by dark magic.

The lead developer decided to store the hitpoints of all the monsters in an array

monsters: Array[Integer]

but gave you the responsibility to efficiently operate on the array:

- attack(x): remove x hitpoints from the enemy with the most remaining hitpoints.
- restore(x): add x hitpoints to the enemy with the most remaining hitpoints.
- hasWon() → Boolean: are there **any** **no** enemies with positive hitpoints left?

Initially, all hitpoint numbers are positive and the array is sorted in descending order (so the monster with the most hitpoints is at position 0). But you do not need to keep the array sorted: all that matters is that it always contains the current hitpoint numbers in some order.

- Describe an invariant on the array that helps you implement the operations efficiently.
- Implement the three operations (maintaining your invariant, of course).
- State the asymptotic complexity of each function in terms of the number N of enemies.

For full points, two of the operations should be $O(1)$ and all should be faster than $O(N)$.

Note: You can freely use operations on data structures and algorithms from the course – you do not have to explain how they work.

Answer

a) **Invariant: the array is a binary max-heap. Note that this holds in the initial state.**

b) **attack(x):**
 monsters[0] = monsters[0] - x
 sink(monsters, 0) **// also called sift_down or bubble_down**

restore(x):
 monsters[0] = monsters[0] + x

hasWon() → Boolean:
 return monsters[0] ≤ 0 **// or negated (as per the specification)**

c) **attack is $O(\log(N))$, but restore and hasWon are $O(1)$.**

Here is a reminder how sink works (not required):

```
sink(array, node):
    largest = node
    for child in [node · 2 + 1, node · 2 + 2]: // the two possible children
        if child < array.length and array[child] > array[largest]:
            largest = child
    if largest ≠ node:
        swap node and largest in array
        sink(array, largest)
```

CHALMERS	Anonymous code		Consecutive page no Löpande sid nr
	Anonym kod		Question no. Uppgift nr 10

Advanced question 9: Cursed trains

You are given a type S of *stations* and a type T of *trains* (both supporting $O(1)$ comparison).

Each train is cursed to forever cycle through a list of stations:

```
stations_for_train: Map[T, List[S]]           // O(1) get
```

From this, you have already computed which trains stop at a given station:

```
trains_for_station: Map[S, List[T]]           // O(1) get
```

You want to get from station A to station B by taking the fewest number of these cursed trains. Find an algorithm for this that runs in $O(N \log(N))$ where N is the total size of the above lists.

Note: You can freely use data structures and algorithms from the course – you do not have to explain how they work.

Hint: How can you model getting on or getting off a train?

Answer

There are several workable solution ideas. Here is the simplest one.

We implement a graph with nodes the **disjoint union** $S \sqcup T$. So a node is either a station or a train.

- In Python or Haskell, this is denoted $S \mid T$.
- In Java, one would create an interface `Node` with classes `SNode` and `TNode`.
- Using null values, one can also hack this as a class with fields for both S and T , of which exactly one is null (not recommended).

The outgoing edges method is a simple wrapper:

```
outgoing_edges(x: S  $\sqcup$  T)  $\rightarrow$  List[S  $\sqcup$  T]:
  if x in S: return trains_for_stations.get(x)
  if x in T: return stations_for_trains.get(x)
```

Note that this graph has N edges.

We now simply run an ordinary breadth-first search from node A to node B . This is $O(N)$. It works because every station is only connected to trains and vice versa, so the shortest unweighted path between two stations will be one that involves stepping on/off as few trains as possible.

Note: It might be tempting to just do a standard graph search in the graph whose nodes are stations with an edge between two nodes if there is a train that connects them. However, this graph has $O(N^2)$ edges (a train serving K stations turns into $K(K-1)$ edges). So you would need to tweak the algorithm to make sure that there is no quadratic blow-up.